

Manual de Usuario — spaCEinvaders

Instituto: Instituto Tecnológico de Costa Rica **Escuela:** Ingeniería en Computación

Curso: CE3104— Paradigmas de Programación

Proyecto: spaCEinvaders — Implementación de Space Invaders con paradigmas imperativo y orientado a objetos

Desarrollador: Jestim Espinoza Morales

Carné: 2023282666

Introducción

spaCEinvaders es una recreación del clásico arcade *Space Invaders*, desarrollada como proyecto del curso CE3104. La aplicación está dividida en dos componentes que se comunican entre sí por red:

- Un **servidor en Java**, que concentra toda la lógica del juego: movimiento de los extraterrestres, disparos, colisiones, puntuación, oleadas y el estado de cada partida. El servidor es la única fuente de verdad — ninguna decisión de juego se toma en el cliente.
- Un **cliente gráfico en C**, construido con la librería Raylib, que se conecta al servidor mediante sockets TCP, envía las acciones del jugador (mover el cañón, disparar) y dibuja en pantalla el estado que recibe del servidor en cada ciclo.

Además, el servidor incluye una **consola de administración gráfica** (Swing) que permite observar todas las partidas activas en tiempo real y enviar comandos especiales (crear aliens, lanzar un OVNI, ajustar la velocidad, dañar los bunkers, etc.) sin necesidad de tocar el código.

1. Requisitos del sistema

Antes de compilar o ejecutar el proyecto, es necesario verificar que el equipo cuenta con las siguientes herramientas y recursos. La tabla resume los requisitos mínimos; en la práctica, cualquier equipo moderno (de los últimos 5-6 años) los cumple sin problema.

Componente	Requisito mínimo
Sistema operativo	Windows 10 (64 bits) / Linux con GCC y librerías estándar
Java Runtime Environment	JDK 11 o superior

Componente	Requisito mínimo
CMake	3.14 o superior (para compilar el cliente C)
Compilador C	GCC/MinGW compatible con C99
Conectividad de red	Interfaz de red loopback (127.0.0.1) disponible
Memoria RAM	256 MB mínimo
Resolución de pantalla	900 × 680 píxeles o superior

2. Estructura de archivos del proyecto

Conocer la organización de carpetas ayuda a ubicar rápidamente el código fuente, los archivos de configuración de compilación y la documentación.

```

tarea ImPoo/
├── CMakeLists.txt           - configuración de compilación del cliente C
├── pom.xml                 - configuración Maven para el servidor Java
├── server/src/com/spaceinvaders/
│   ├── model/              - modelos del dominio (Jugador, Extraterrestre, Bunker, etc.)
│   ├── factory/            - fábricas de aliens (Abstract Factory)
│   ├── observer/           - interfaces y notificador (Observer)
│   ├── network/            - manejadores de conexiones TCP
│   └── server/              - motor del juego, servidor y consola admin
├── client/
│   ├── include/            - cabeceras del cliente C
│   └── src/                 - código fuente del cliente C con Raylib
└── docs/
    ├── documentacion.pdf
    └── Manual_de_Usuario.pdf

```

A grandes rasgos: todo lo que vive bajo `server/` se compila con Java/Maven, todo lo que vive bajo `client/` se compila con CMake/GCC, y `docs/` contiene la documentación del proyecto (incluyendo este manual).

3. Compilación del servidor Java

El servidor se compila con Maven desde la raíz del proyecto. Maven se encarga de resolver dependencias, compilar las clases y dejar los `.class` listos para ejecutarse, todo con un solo comando:

```
mvn compile
```

O bien desde el IDE IntelliJ IDEA ejecutando la configuración de build estándar. Las clases compiladas quedan en `target/classes/`.

Para ejecutar pruebas unitarias (recomendado después de cualquier cambio al código del servidor, para confirmar que la lógica de juego sigue comportándose como se espera):

```
mvn test
```

4. Compilación del cliente C (Windows)

El cliente C utiliza CMake y depende de Raylib, que se descarga automáticamente mediante FetchContent en la primera compilación (requiere conexión a internet). Esto significa que la primera vez que se compila el proyecto puede tardar más de lo normal, mientras CMake descarga y compila Raylib desde su repositorio oficial; las compilaciones siguientes son mucho más rápidas porque Raylib queda cacheado localmente.

Desde CLion 2026.1.1 (método recomendado):

1. Abrir CLion y seleccionar la carpeta raíz del proyecto.
2. CLion detecta automáticamente `CMakeLists.txt`.
3. En el selector de configuración (esquina superior derecha), seleccionar `spaceinvaders`.
4. Hacer clic en el botón ► Run (o presionar Shift+F10).

Este método es el recomendado porque CLion ya incluye una distribución de MinGW configurada correctamente, evitando el problema descrito en la nota más abajo.

Desde terminal (PowerShell):

```
$env:PATH = "C:\Program Files\JetBrains\CLion 2026.1.1\bin\mingw\bin;$env:PATH"
$cmake = "C:\Program Files\JetBrains\CLion 2026.1.1\bin\cmake\win\x64\bin\cmake.exe"
```

```
$p = Start-Process -FilePath $cmake `
    -ArgumentList "--build", "cmake-build-debug", "--target", "spaceinvaders" `
    -WorkingDirectory "C:\Users\Usuario I 2026\Desktop\tarea ImPoo" `
    -NoNewWindow -Wait -PassThru `
    -RedirectStandardOutput "C:\Temp\build_out.txt" `
    -RedirectStandardError "C:\Temp\build_err.txt"
```

```
Get-Content C:\Temp\build_out.txt
Get-Content C:\Temp\build_err.txt
```

Nota importante: Es obligatorio incluir el directorio bin de MinGW en el PATH antes de invocar CMake. Sin este paso, `cc1.exe` no encuentra sus DLLs y GCC falla

silenciosamente con código de salida 1 sin producir ningún mensaje de error. Este es uno de los problemas más difíciles de diagnosticar si no se conoce la causa de antemano, porque la compilación simplemente “no hace nada” en lugar de mostrar un error claro.

El binario resultante queda en `cmake-build-debug\spaceinvaders.exe`.

5. Ejecución de la aplicación

La aplicación consta de tres componentes que deben iniciarse en el siguiente orden: primero el servidor (que debe estar escuchando antes de que cualquier cliente intente conectarse), y después uno o más clientes.

Paso 1: Iniciar el servidor Java

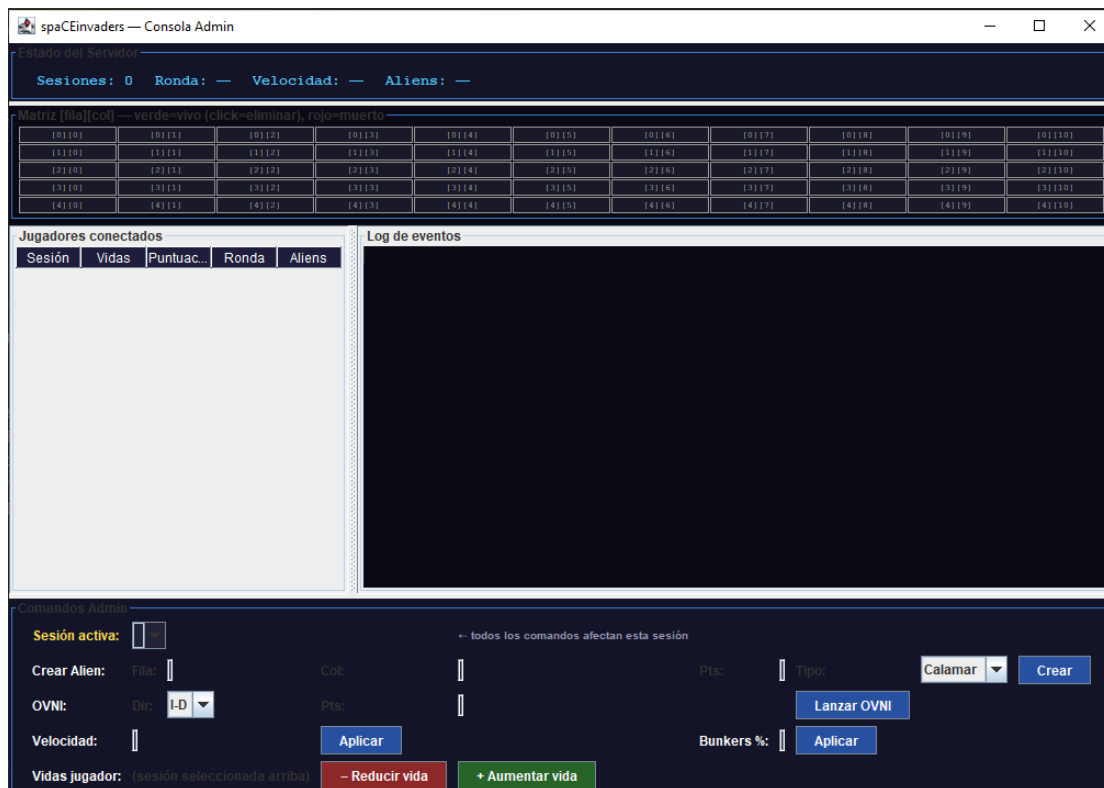
Desde IntelliJ IDEA: ejecutar la clase ServidorJuego.java

O desde terminal:

```
java -cp target/classes com.spaceinvaders.server.ServidorJuego
```

Al iniciarse, el servidor abre dos puertos de escucha: - Puerto **8080**: conexiones de jugadores. - Puerto **8081**: conexiones de espectadores (solo lectura).

La interfaz gráfica de administración (ConsolaAdmin) aparece automáticamente al iniciar el servidor — no es necesario abrirla por separado ni escribir comandos en la consola de texto, aunque esta última sigue funcionando como alternativa.



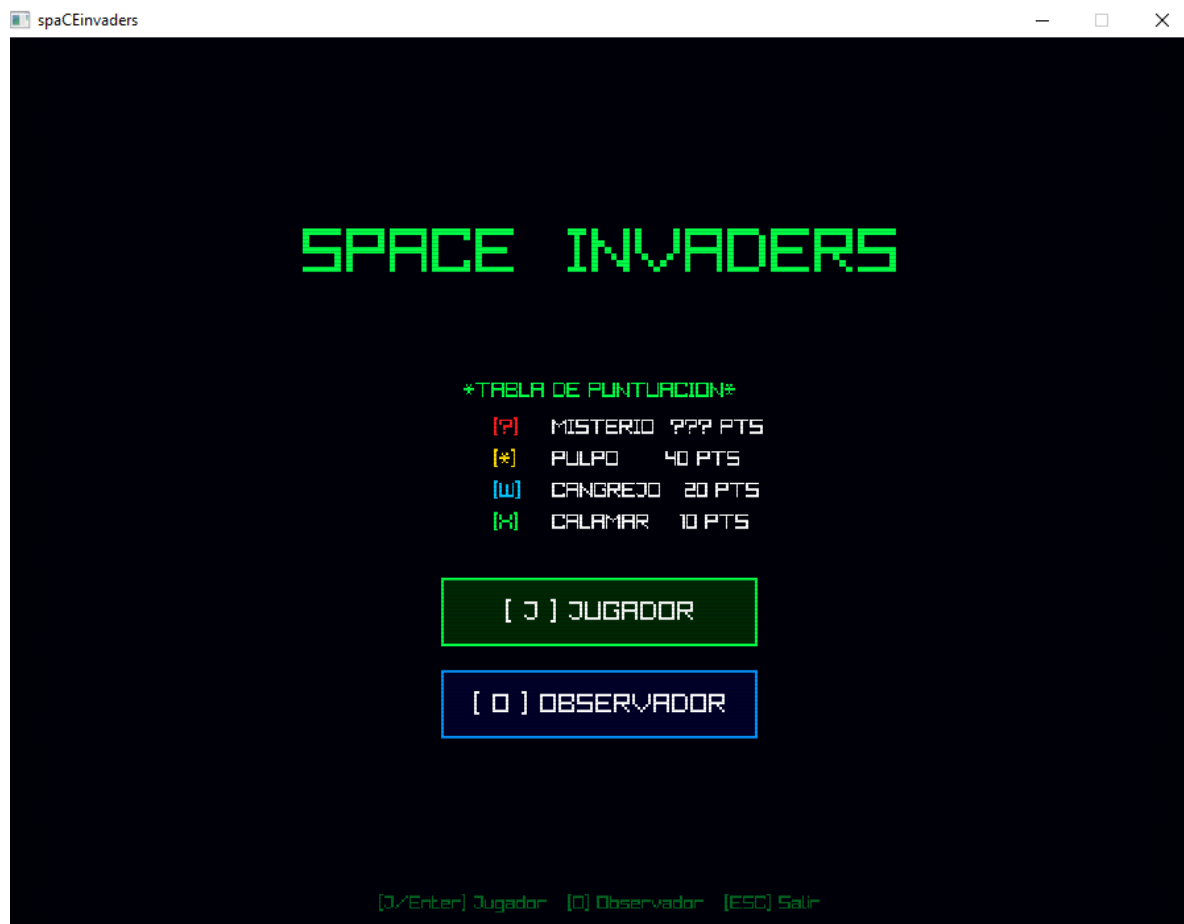
Paso 2: Ejecutar el cliente C

`cmake-build-debug\spaceinvaders.exe`

Al iniciar, el cliente muestra el menú principal con dos opciones:

Tecla	Acción
J o Enter	Conectar como Jugador (puerto 8080)
O	Conectar como Observador (puerto 8081)

El menú también acepta clic de mouse sobre cada opción, como alternativa al teclado.

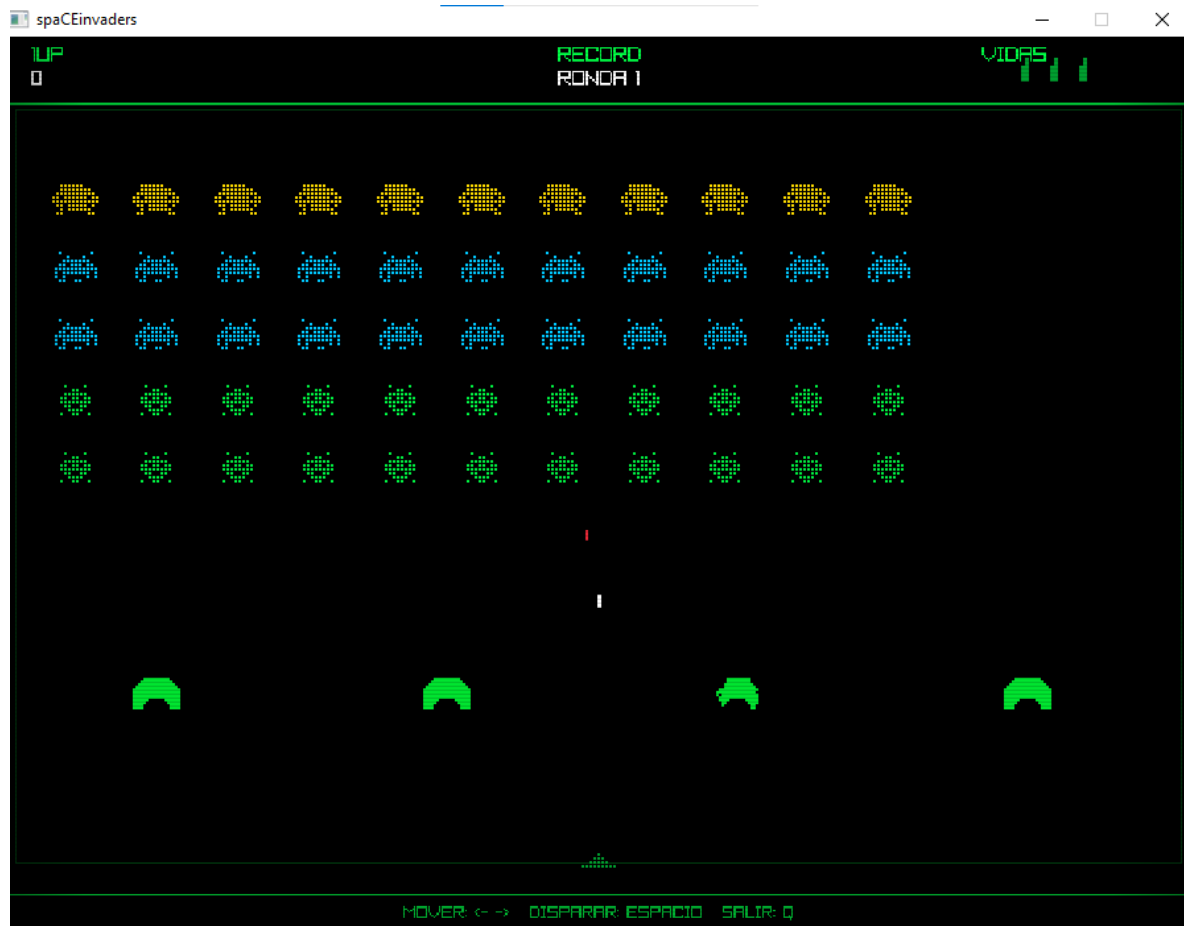


6. Controles del jugador

Una vez dentro de una partida como jugador, los controles disponibles son los siguientes:

Tecla	Acción
← (flecha izquierda)	Mover el cañón hacia la izquierda
→ (flecha derecha)	Mover el cañón hacia la derecha

Tecla	Acción
Espacio	Disparar
Q	Salir del juego
R o Enter (pantalla fin)	Reiniciar partida
S o Escape (pantalla fin)	Salir al menú



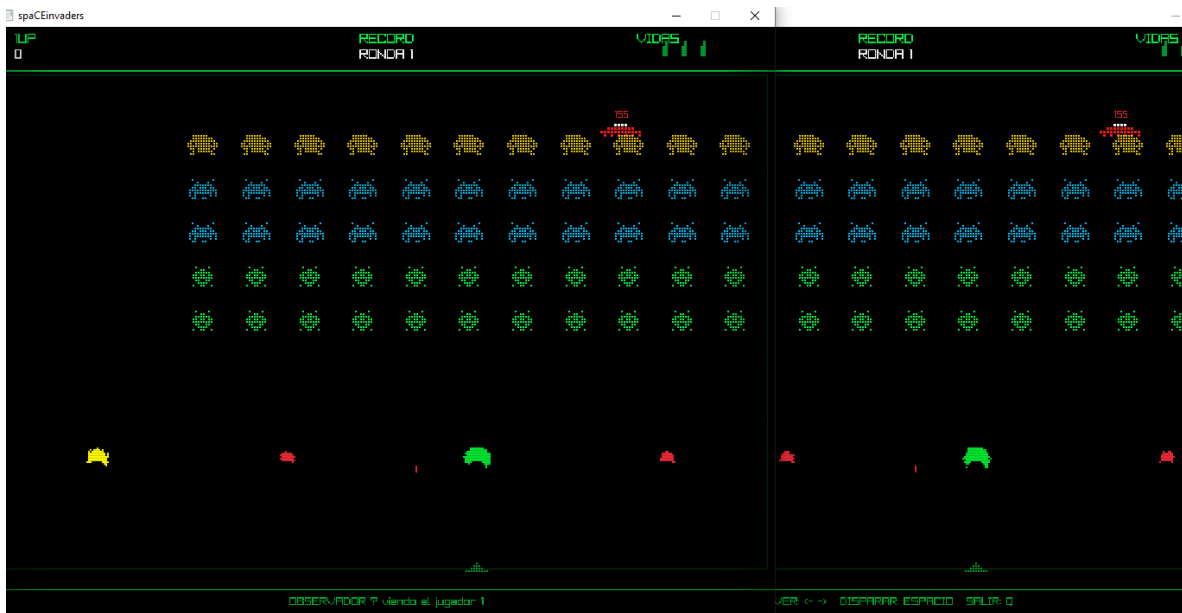
Cuando la partida termina (por pérdida de todas las vidas o por decisión del jugador), aparece una pantalla final con las opciones de reiniciar o salir al menú, descritas en la tabla anterior.



7. Modo espectador

Al seleccionar 0 en el menú, el cliente se conecta al puerto 8081 y muestra la lista de partidas activas numeradas. Este modo es útil, por ejemplo, para que una persona observe la partida de otra sin interferir, o para verificar visualmente el efecto de los comandos enviados desde la consola de administración.

El espectador selecciona una partida con las teclas 1-8 o haciendo clic en el botón correspondiente. Una vez seleccionada, observa la partida en tiempo real sin posibilidad de enviar comandos — el cliente en modo espectador recibe exactamente el mismo flujo de estado que el cliente jugador, pero nunca envía mensajes de movimiento o disparo.



8. Consola de administración (interfaz gráfica del servidor)

La ventana de administración se abre automáticamente con el servidor y es la herramienta principal para supervisar y manipular las partidas en curso sin necesidad de modificar código ni reiniciar el servidor. Permite:

- **Visualizar** el estado de todas las sesiones activas (aliens vivos, ronda, velocidad, jugadores y sus puntuaciones).
- **Matriz 5×11 de aliens:** muestra en verde las celdas con alien vivo y en rojo las destruidas. Hacer clic en una celda activa destruye ese alien — útil para probar manualmente la lógica de fin de ronda sin tener que jugar hasta el final.
- **Selector de sesión:** permite elegir a cuál sesión (de entre todas las partidas activas) aplicar los comandos siguientes, de modo que cada partida pueda manipularse de forma independiente.
- **Comandos disponibles:**

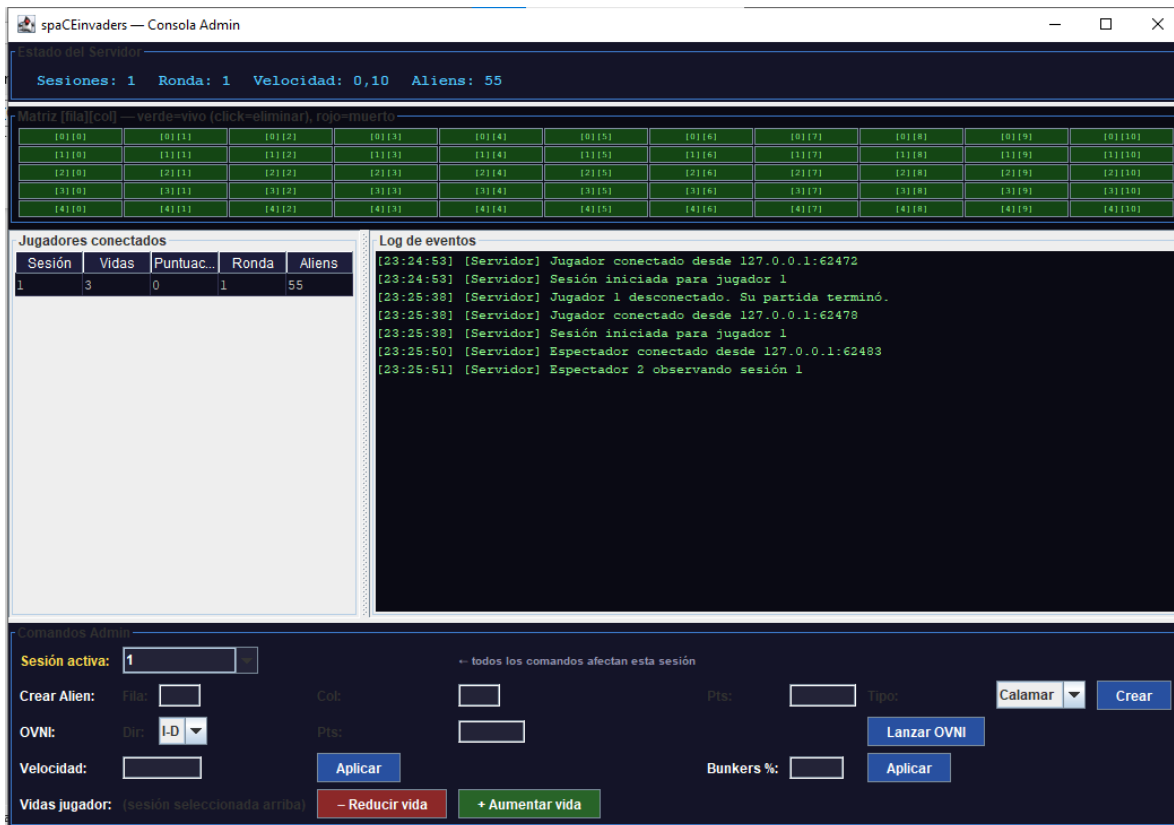
Campo	Descripción	Ejemplo
Fila + Columna + Puntos + Tipo	Crear alien en posición de la matriz	Fila=0, Col=5, Pts=40, Tipo=Pulpo
Dirección OVNI + Puntos	Lanzar OVNI	Dir=I-D, Pts=1500
Velocidad	Ajustar velocidad de los aliens	2.0
Bunkers %	Establecer porcentaje de salud de los bunkers	70
Reducir vida	Quita una vida al jugador de la sesión seleccionada	—
Aumentar vida	Agrega una vida al jugador	—

Campo

Descripción

Ejemplo

de la sesión seleccionada



9. Protocolo de mensajes (referencia)

Esta sección documenta el formato exacto de los mensajes JSON que viajan entre servidor y cliente, útil principalmente para quien desee depurar la comunicación de red o extender el protocolo. No es necesario conocer este detalle para simplemente jugar.

Servidor → cliente (emitido cada 50 ms):

```
{
  "tipo": "ESTADO_JUEGO",
  "vidas": 3,
  "puntuacion": 0,
  "velocidad": 1.0,
  "ronda": 1,
  "canonX": 50,
  "balaActiva": false,
  "balaX": 0,
  "balaY": 85,
  "aliens": [100, 5, 10, 2, 40, 101, 14, 10, 2, 40],
  "bunkers": [100, 75, 50, 25],
  "ovni": [0, 0, 0],
```

```
"balasAlien": [30, 45, 60, 55]
}
```

Los arrays son planos de enteros: aliens agrupa 5 valores por alien (id, x, y, tipo, puntos); ovni agrupa 3 valores (activo, x, puntos); balasAlien agrupa 2 valores por bala (x, y). Este diseño de “arrays planos” en lugar de listas de objetos JSON anidados fue elegido deliberadamente para reducir el tamaño de cada mensaje, ya que el servidor envía este paquete 20 veces por segundo a cada cliente conectado.

Cliente → servidor:

```
{"tipo": "MOVER_CANON", "direccion": "IZQUIERDA"}
{"tipo": "MOVER_CANON", "direccion": "DERECHA"}
{"tipo": "DISPARO"}
```

Estos mensajes se envían únicamente cuando ocurre el evento correspondiente (una pulsación de tecla), nunca de forma continua — esto evita saturar la conexión con mensajes redundantes mientras una tecla permanece presionada.

Fin del Manual de Usuario. Para detalles sobre las estructuras de datos internas, los algoritmos del motor de juego, los problemas encontrados durante el desarrollo y las conclusiones del proyecto, consultar [documentacion.pdf](#).